Práctica 5: Parsers LR(0) y SLR(1)
Construcción de Tablas y Motor de Ejecución

Laboratorio de Compiladores

1. Objetivo

Comprender la mecánica de los analizadores sintácticos ascendentes (Bottom-Up) mediante la construcción manual del autómata LR(0) y sus tablas de análisis (Acción y Goto). Implementar el motor de ejecución (driver) basado en pila para un parser LR en el lenguaje de programación de su elección, identificando de manera práctica cómo el uso de los conjuntos de anticipación (Follow) en SLR(1) resuelve los conflictos de desplazamiento/reducción inherentes a LR(0).

2. Conceptos Base

2.1. Reglas para First y Follow

Para construir las tablas SLR(1) es indispensable conocer el contexto de cada no terminal.

- **First(α):** Conjunto de terminales que inician las cadenas derivadas de α.
  1. Si X es terminal, First(X) = $\{X\}$.
  2. Si existe $X \to \epsilon$, entonces $\epsilon \in$ First(X).
  3. Si $X \to Y_1 Y_2 \dots Y_k$, se añade First(Y_1)$\setminus\{\epsilon\}$ a First(X). Si $Y_1 \Rightarrow^* \epsilon$, se añade First($Y_2$)$\setminus\{\epsilon\}$, y así sucesivamente.
- **Follow(A):** Conjunto de terminales que pueden aparecer inmediatamente a la derecha de A.
  1. Para el símbolo inicial S, añadir \$ a Follow(S).
  2. Si existe una producción $B \to \alpha A \beta$, añadir First(β) $\setminus \{\epsilon\}$ a Follow(A).
  3. Si existe $B \to \alpha A$, o $B \to \alpha A \beta$ donde $\beta \Rightarrow^* \epsilon$, añadir todo Follow($B$) a Follow($A$).

2.2. Construcción del Autómata LR(0)

Algorithm 1 Cerradura(I) y Goto(I, X)

function CERRADURA(I)
  $J \leftarrow I$
  repeat
    for cada ítem $[A \to \alpha \cdot B\beta]$ en J **do**
      for cada producción $B \to \gamma$ de la gramática **do**
        if $[B \to \cdot\gamma]$ no está en J **then**
          Añadir $[B \to \cdot\gamma]$ a J
        end if
      end for
    end for
  until no se añadan más ítems a J
  return J
end function

function GOTO(I, X)
  $J \leftarrow \emptyset$
  for cada ítem $[A \to \alpha \cdot X\beta]$ en I **do**
    Añadir $[A \to \alpha X \cdot \beta]$ a J
  end for
  return CERRADURA(J)
end function

2.3. Construcción de las Tablas LR(0) y SLR(1)

Algorithm 2 Llenado de Tablas de Acción y Goto

for cada estado I_i del autómata **do**
  1. Desplazamiento (Shift): Si $[A \to \alpha \cdot a\beta] \in I_i$ (a es terminal) y Goto$(I_i, a) = I_j$, establecer Accion$[i, a]$ = shift j.
  2. Goto: Si Goto$(I_i, A) = I_j$ (A es no terminal), establecer Goto$[i, A] = j$.
  3. Aceptación: Si $[S' \to S\cdot] \in I_i$, establecer Accion$[i, \$]$ = accept.
  4. Reducción (Reduce): Si $[A \to \gamma\cdot] \in I_i$ corresponde a la regla k:
  if Es LR(0) then
    Para **todos** los terminales t (incluyendo \$), Accion$[i, t]$ = reduce k.
  else if Es SLR(1) then
    Para todo terminal $t \in$ **Follow**(A), Accion$[i, t]$ = reduce k.
  end if
end for

3. Desarrollo

3.1. 1. Definición de la Gramática

Cada equipo deberá diseñar una gramática libre de contexto que cumpla con las siguientes características:

- Contener al menos **20 producciones** (reglas).
- Debe modelar una estructura de lenguaje real (ej. declaraciones de variables, expresiones aritméticas/lógicas sencillas, bloques de código, etc.).

- **Pauta crítica:** La gramática debe presentar **conflictos de desplazamiento/reducción en LR(0)** que se resuelvan exitosamente al aplicar los conjuntos Follow para convertirla a **SLR(1)**.
- *Tip:* Para lograr esto fácilmente, incluyan estructuras jerárquicas de expresiones (como sumas y multiplicaciones con precedencia) o reglas recursivas por la izquierda y derecha sobre los mismos símbolos.

Publicación: Para evitar gramáticas duplicadas, un representante de cada equipo debe publicar su gramática en los comentarios generales de la asignación en Classroom. Las gramáticas operan bajo la regla de *"primero en publicar, primero en reservar"*.

3.2. 2. Trabajo Analítico (A mano)

Deberán calcular manualmente el proceso de obtención de tablas:

1. Calcular los conjuntos **First** y **Follow** de todos los no terminales.
2. Construir la colección canónica de conjuntos de ítems (Autómata LR(0)) mostrando las transiciones Goto.
3. Construir la matriz de análisis sintáctico. Demostrar explícitamente en qué celdas ocurre el conflicto LR(0) y cómo la regla de SLR(1) filtra las reducciones para dejar una tabla limpia.

3.3. 3. Implementación del Motor LR (Driver)

En el lenguaje de su elección (no hay plantillas en esta práctica), deberán programar el **motor de ejecución del parser LR**.

- Las tablas SLR(1) obtenidas manualmente deben estar **codificadas directamente** (hard-codeadas) como matrices o diccionarios en su código. ¡No programen el generador de tablas, solo el intérprete de la matriz!
- El programa recibirá una lista de tokens (cadena de entrada).
- Implementará la pila clásica de estados y símbolos.
- Ejecutará el ciclo de shift/reduce basado en la tabla hasta llegar a *Accept* o Error.
- Como salida, el programa indicará si la cadena es válida y mostrará el **AST (Abstract Syntax Tree) en modo texto** (puede ser imprimiendo las reglas utilizadas en las reducciones con su debida tabulación/identación jerárquica).

4. Especificaciones de Entrega

El reporte debe entregarse en un único archivo PDF. **NO incluyan capturas de pantalla de código.** Se evaluará la capacidad analítica y la limpieza de la solución. El reporte debe contener exclusivamente:

1. **Gramática Propuesta:** La lista de producciones numeradas y los tokens definidos.
2. **Proceso de Tablas y Resolución SLR(1):** El desarrollo manual (First, Follow, Autómata y Tablas). Debe haber un párrafo corto demostrando matemáticamente cómo SLR(1) resolvió las ambigüedades de LR(0) basándose en las celdas conflictivas.

3. **Diseño de Solución:** Un diagrama de flujo o diagrama de arquitectura (ej. UML sencillo o esquema de bloques) que ilustre la lógica implementada para la máquina de pila (driver) y cómo consulta la matriz.

4. **Tabla de Pruebas y Resultados:** Una tabla formal con al menos 10 cadenas de prueba (5 válidas, 5 inválidas). Indicar para cada una los tokens de entrada, si fue aceptada o rechazada, y un breve comentario sobre por qué.

5. **Enlace al Repositorio:** El código completo debe subirse a Git (GitHub, GitLab, etc.). En el reporte solo coloquen el enlace público al repositorio. Este código servirá como evidencia de autoría y para consultas en caso de dudas, pero **no** se requiere incluir bloques de código en el PDF.

5. Evaluación de la Práctica

Instrucciones: Indique su grado de acuerdo con cada una de las siguientes afirmaciones utilizando una escala de 1 a 5, donde: 1. Totalmente en desacuerdo 2. En desacuerdo 3. Neutral 4. De acuerdo 5. Totalmente de acuerdo

Criterio / Afirmación	**1**	**2**	**3**	**4**	**5**
El calcular manualmente el autómata me ayudó a entender por qué LR(0) falla en la vida real.					
Programar el motor de la pila por separado de la tabla dejó más clara la arquitectura de un compilador.					
El nivel de complejidad técnica fue adecuado y el no usar plantillas permitió mayor libertad creativa en el código.					
Las pautas para definir la gramática fueron claras y evitaron bloqueos durante el diseño.					
Considero que separar el análisis teórico (a mano) del código fuente redujo mi frustración frente a la computadora.					

Comentarios y Retroalimentación

Por favor, dedica un par de líneas para comentar:

- ¿Qué aspecto de esta práctica fue tu favorito o te resultó más revelador?
- ¿Qué cambiarías o mejorarías de esta práctica para futuras generaciones?

(Espacio para respuesta...)